```
        }
      }
    }
  }
}
```

We are, in effect, replacing the `standard` analyzer, which is the default for a `keyword` analyzer. We can test the analyzer by invoking the `_analyze` endpoint on the index.

Listing 7.29 Testing the default analyzer

```
PUT authors_with_default_analyzer/_analyze
{
  "text":"John Doe"
}
```

This code outputs a single token `"John Doe"` with no lowercasing or tokenizing, indicating that our `keyword` analyzer has analyzed it. You can try the same code using a `standard` analyzer, and you'll notice the difference.

Setting analyzers at an index level or field level works during the indexing process. However, we can also use a different analyzer during the querying process—let's see why and how.

7.5.2 Analyzers for searching

Elasticsearch lets us specify a different analyzer during query time than during indexing. Let's see these two methods in this section as well as some rules that Elasticsearch follows when picking up an analyzer defined at various levels.

Analyzer in a query

We haven't looked in depth at search yet, so don't worry if the following listing baffles you a bit (we discuss search in the next few chapters).

Listing 7.30 Setting the analyzer alongside a search query

```
GET authors_index_for_search_analyzer/_search
{
  "query": {
    "match": {
      "author_name": {
        "query": "M Konda",
        "analyzer": "simple"
      }
    }
  }
}
```

Query to search all authors with the given criteria

The analyzer is specified explicitly and is usually different than the one the field was indexed with.

We specify the analyzer explicitly (the `author_name` field was probably indexed using a different type of analyzer!) while searching for an author.